„UNIVERSITATEA TEHNICĂ A MOLDOVEI
FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ
DEPARTAMENTUL INGINERIA SOFTWARE ȘI AUTOMATICĂ

Lucrare de laborator Nr. 4

Criptografie

Cifruri bloc. Algortimul DES

Realizat: **Crupa Gheorghe, st. gr. SI-266**

Verificat: **Maia ZAICA, asist. univ.**

Chișinău, 2026

Scopul lucrării

Studierea principiilor de funcționare ale cifrurilor bloc simetrice și implementarea practică într-un limbaj de programare a unui element din cadrul algoritmului DES (Data Encryption Standard), cu afișarea la ecran a tabelelor utilizate și a tuturor pașilor intermediari.

Considerații teoretice

Algoritmul DES (Data Encryption Standard) este un cifru bloc simetric care criptează blocuri de date de 64 de biți utilizând o cheie de 56 de biți, având la bază o rețea Feistel cu 16 runde iterative.

După aplicarea permutării inițiale (IP), blocul de 64 de biți este împărțit în două jumătăți a câte 32 de biți: jumătatea stângă L0 și jumătatea dreaptă R0. Pe parcursul celor 16 runde (de la 1 la 16), blocurile se prelucrează după regulile structurii Feistel:

- Li = Ri-1
- Ri = Li-1 + f(Ri-1, Ki), unde semnul plus reprezintă adunarea modulo 2 bit cu bit (operația logică XOR).

În interiorul funcției de rundă f, după expansiunea blocului Ri-1 de la 32 la 48 de biți și aplicarea operației XOR cu subcheia rundei Ki, rezultatul este împărțit în 8 grupuri de câte 6 biți (B1, B2, B3, B4, B5, B6, B7, B8). Fiecare grup este trecut prin cutia de substituție corespunzătoare S-Box, reducându-se la 4 biți, obținându-se în total 32 de biți: S1(B1)S2(B2)...S8(B8). Asupra acestui rezultat se aplică permutarea P pentru a obține valoarea finală a funcției f = P(S1(B1)...S8(B8)), care ulterior este combinată prin XOR cu Li-1 pentru a determina blocul Ri.

Desfășurarea lucrării (Sarcina 2.10)

Conform numărului de ordine din lista grupei (n = 10) și formulei nr_sarcina = n mod 11, a fost selectată Sarcina 2.10: cunoscând în runda i a algoritmului DES ieșirea cutiilor-S S1(B1)...S8(B8) și valoarea blocului L(i-1) (32 de biți), să se calculeze blocul Ri.

Pentru implementare s-a utilizat limbajul Python. Programul permite generarea aleatorie a celor două șiruri de 32 de biți, afișând tabelul de permutare P, rezultatul intermediar f și calculul XOR bit cu bit pentru obținerea lui Ri.

Codul sursă integral al proiectului a fost versionat și poate fi accesat pe platforma GitHub la următoarea adresă: https://github.com/gsohptgs/Criptografie_Lucrari_de_Laborator.git

Codul sursă (Python):

```
import random
# Tabelul de permutare P conform standardului DES
P_TABLE = [
    16,  7, 20, 21,
    29, 12, 28, 17,
     1, 15, 23, 26,
     5, 18, 31, 10,
     2,  8, 24, 14,
    32, 27,  3,  9,
    19, 13, 30,  6,
    22, 11,  4, 25
]

def format_biti(sir):
    """Grupează biții câte 4 pentru o afișare clară la ecran."""
    return ' '.join(sir[i:i+4] for i in range(0, len(sir), 4))

def permutare_p(bloc_32):
    """Aplică tabelul de permutare P asupra blocului de 32 de biți."""
    return ''.join(bloc_32[pozitie - 1] for pozitie in P_TABLE)

def xor_biti(sir1, sir2):
    """Aplică operația XOR bit cu bit (adunare modulo 2)."""
    return ''.join('1' if b1 != b2 else '0' for b1, b2 in zip(sir1, sir2))

def des_sarcina_2_10():
    print("=== LUCRARE DE LABORATOR NR. 4: ALGORITMUL DES (Sarcina 2.10) ===")
    print("1 - Generare aleatorie a datelor (Recomandat - rapid)")
    print("2 - Introducere manuala de la tastatura")

    optiune = input("Alege modul de introducere a datelor (1 sau 2): ").strip()

    if optiune == "1":
        s_output = ''.join(random.choice('01') for _ in range(32))
        l_anterior = ''.join(random.choice('01') for _ in range(32))
    elif optiune == "2":
        s_output = input("Introdu S1(B1)...S8(B8) (exact 32 de biti de 0 si 1): ").replace(" ",
"")
        l_anterior = input("Introdu L_(i-1) (exact 32 de biti de 0 si 1): ").replace(" ", "")
        if len(s_output) != 32 or len(l_anterior) != 32 or not set(s_output +
l_anterior).issubset({'0', '1'}):
            print("Eroare: Datele trebuie sa contina exact 32 de caractere binare (0 sau 1)!")
            return
    else:
        print("Optiune invalida!")
        return

    print("\n--- 1. DATE DE INTRARE ---")
    print(f"S1(B1)...S8(B8) : {format_biti(s_output)}")
    print(f"L_(i-1)         : {format_biti(l_anterior)}")

    print("\n--- 2. TABELUL DE PERMUTARE P UTILIZAT ---")
    for i in range(0, 32, 4):
        print(f"  {P_TABLE[i]:2d}  {P_TABLE[i+1]:2d}  {P_TABLE[i+2]:2d}  {P_TABLE[i+3]:2d}")

    f_rezultat = permutare_p(s_output)
    print("\n--- 3. PAS INTERMEDIAR: APLICAREA PERMUTARII P ---")
    print("Formula: f = P(S1(B1)...S8(B8))")
    print(f"Rezultat f      : {format_biti(f_rezultat)}")

    r_curent = xor_biti(l_anterior, f_rezultat)
    print("\n--- 4. PAS FINAL: CALCULUL LUI R_i (OPERATIA XOR) ---")
    print("Formula: R_i = L_(i-1) XOR f")
    print(f"  {format_biti(l_anterior)}  (L_(i-1))")
    print(f"⊕ {format_biti(f_rezultat)}  (f)")
    print("  " + "-" * 39)
    print(f"  {format_biti(r_curent)}  (R_i)")

    print(f"\nREZULTAT FINAL R_i (32 biti): {r_curent}")

des_sarcina_2_10()
```

Datele de testare și rezultatele obținute:

- **Ieșirea cutiilor-S (S1(B1)...S8(B8)):** `1101 0011 1110 0011 0010 0011 1101 1110`
- **Blocul stâng anterior (L_(i-1)):** `1110 1000 0110 0011 0011 0000 1110 0110`
- **Rezultatul intermediar după permutarea P (f):** `1100 1010 1111 0011 1110 0001 1010 0111`
- **Rezultatul final (R_i):** `0010 0010 1001 0000 1101 0001 0100 0001` (`00100010100100001101000101000001`)

```
PS C:\Users\CC\Documents\GitHub\criptografie_lucrari_de_laborator\l4_c_si-266> python des.py
=== LUCRARE DE LABORATOR NR. 4: ALGORITMUL DES (Sarcina 2.10) ===
1 - Generare aleatorie a datelor (Recomandat - rapid)
2 - Introducere manuala de la tastatura
Alege modul de introducere a datelor (1 sau 2): 1

--- 1. DATE DE INTRARE ---
S1(B1)...S8(B8) : 1101 0011 1110 0011 0010 0011 1101 1110
L_(i-1)         : 1110 1000 0110 0011 0011 0000 1110 0110

--- 2. TABELUL DE PERMUTARE P UTILIZAT ---
  16   7  20  21
  29  12  28  17
   1  15  23  26
   5  18  31  10
   2   8  24  14
  32  27   3   9
  19  13  30   6
  22  11   4  25

--- 3. PAS INTERMEDIAR: APLICAREA PERMUTARII P ---
Formula: f = P(S1(B1)...S8(B8))
Rezultat f      : 1100 1010 1111 0011 1110 0001 1010 0111

--- 4. PAS FINAL: CALCULUL LUI R_i (OPERATIA XOR) ---
Formula: R_i = L_(i-1) XOR f
  1110 1000 0110 0011 0011 0000 1110 0110  (L_(i-1))
⊕ 1100 1010 1111 0011 1110 0001 1010 0111  (f)
  -----------------------------------
  0010 0010 1001 0000 1101 0001 0100 0001  (R_i)

REZULTAT FINAL R_i (32 biti): 00100010100100001101000101000001
```

Concluzie

În urma realizării acestei lucrări de laborator, am studiat structura algoritmului de criptare bloc DES și am implementat etapa finală a unei runde Feistel pentru determinarea blocului R_i. Am observat rolul important al permutării P, care dispersează biții rezultați din cutiile de substituție S pe întreg blocul de 32 de biți (asigurând proprietatea de difuzie), precum și rolul operației XOR cu jumătatea stângă L_(i-1), care garantează că procesul este complet reversibil la decriptare prin aplicarea acelorași pași cu subcheile în ordine inversă.